

Task 1 — Web App QA & Debug Report

App under test: **Conduit** (RealWorld “Medium clone”) — Angular frontend + Nitro/Prisma/SQLite API, run locally
Prepared by **Ayush Rawat** · Automation & QA Developer assessment · Tested 29 September 2026

5
High

6
Medium

2
Low

16 / 16
automated checks green (2 checks + 14 repos)

1. Summary for stakeholders

Bottom line: the basic journeys work — sign-up, log-in, writing, editing and deleting an article, commenting and logging out all pass an automated end-to-end test. The app is **not production-ready**, though: I found **13 issues (5 High, 6 Medium, 2 Low)**. Every one was reproduced at least twice and is backed by an automated test, a screenshot or a raw HTTP log.

Why users say it “feels broken” — the three most likely causes

- Any server hiccup freezes the home page** on “Loading articles...” forever — no message, no retry (#1).
- Every edit gives an article a new web address**, so links shared earlier open a **blank white page** (#2, #6).
- Capital letters in an e-mail address lock people out** — “credentials invalid” although the password is right (#8).

Biggest business risk — account security: 1-character passwords are accepted (#5), password guessing is unlimited (#4), and “log out” does not end the session: a copied session token keeps working for 60 days and can even renew itself (#3).

Recommended order of fixes

- Security first:** server-side password/e-mail rules, login rate-limiting and real logout (session revocation, shorter tokens) — #3, #4, #5.
- “Feels broken” fixes:** error + retry states wherever data loads, and a proper “not found” page — #1, #6, #7.
- Stable article URLs** — #2 (a 3-line fix, already verified locally; see §4).
- Input hygiene:** normalise e-mails, restrict usernames, add length/size limits — #8, #9, #10, #12, #13; then accessibility — #11.

2. Scope, approach and environment

App	Conduit, the RealWorld reference app. Frontend: <code>realworld-apps/angular-realworld-example-app @ dd99ed2</code> (Angular 21) — the same frontend that powers the official demo. Backend: <code>realworld-apps/nitro-prisma-zod-realworld-example-app @ c8c6685</code> (Nitro, Prisma 7, SQLite) — one of the two backends the project lists as fully spec-compliant.
Why run it locally	The live-demo link in the brief is dead on the test date (<code>demo.realworld.io</code> → HTTP 404, <code>api.realworld.io</code> → HTTP 530; the project moved to <code>realworld.show</code>). Running locally also keeps test accounts off a shared public service and lets me reset the database so every result is repeatable.
Only change made	The frontend hard-codes <code>https://api.realworld.show/api</code> ; I pointed it at <code>http://localhost:3000/api</code> (one line). Nothing else was modified.
Machine	Windows 11 · Node 24.17 · Google Chrome 154 · API on :3000, frontend on :4200.
Flows covered	Sign-up, log-in, create, edit, delete, comment, favourite/follow, log-out — plus the edge cases suggested in the brief: very long inputs, special characters, simulated network failure, logged-out/expired session, weak credentials.
How	(1) Read the source to form hypotheses → (2) exploratory testing in the browser → (3) API probes (Node script, raw HTTP evidence) → (4) Playwright reproduction test + screenshot for every UI issue → (5) axe-core 4.13 audit against WCAG 2.1 AA. Final run on a fresh database: 16/16 green .
Severity scale	Critical outage / data breach / data loss for many users, no workaround · High core journey broken or serious security weakness · Medium feature partly broken or confusing, workaround exists · Low minor, cosmetic or API-only.

3. Issues found (template from section 5 of the brief)

Ordered by severity. **Evidence:** A1–A12 = screenshots in Appendix A · BUG-xx = Playwright test in `qa-tests/tests` · Pxx = API probe in `evidence/api-probes.json`. File references point to the tested commits.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Home feed never finishes loading when the API has a problem Error handling A1 · BUG-01	1. Make the API unavailable (stop it, or DevTools › Network › Offline). 2. Open the home page. 3. Wait 10+ seconds.	Expected: a clear “Couldn’t load articles” message with a Retry button (and/or automatic retry). Actual: “Loading articles...” and “Loading tags...” stay forever. No error, no retry — only a manual reload helps.	High	<code>ArticleListComponent.runQuery()</code> subscribes with a success handler only (<code>article-list.component.ts:124</code>). The API error is never caught, so the loading state is never cleared and nothing is shown.
2	Editing an article changes its web address — links shared earlier stop working Functional A2–A3 · BUG-02 · P8 · fix verified (\$4)	1. Log in, publish an article, copy its URL. 2. “Edit Article”, change one word in the <i>body</i> only, “Publish Article”. 3. Compare URLs; open the copied one.	Expected: URL unchanged when the title is unchanged; old links keep working. Actual: every save creates a new random URL suffix; the old URL returns 404 and shows a blank page (see #6).	High	The API regenerates the slug whenever a <code>title</code> field is present in an update (<code>articles/[slug]/index.put.ts:35</code>), and the editor always re-sends the unchanged title (<code>editor.component.ts:80</code>). Full analysis in \$4.
3	Logging out does not end the session; tokens live 60 days and renew themselves Security BUG-03 · P6	1. Log in; copy <code>jwtToken</code> from DevTools › Local Storage. 2. Settings › “Or click here to logout”. 3. <code>GET /api/user</code> with header <code>Authorization: Token <copied></code> .	Expected: 401 — token revoked at logout; short-lived tokens. Actual: 200 OK, and the reply contains a <i>fresh</i> 60-day token. A leaked token (shared PC, logs, XSS) gives indefinite access.	High	Logout only deletes the token in the browser (<code>user.service.ts:85</code>); no server-side session or deny-list; <code>expiresIn: '60d'</code> (<code>generate-token.ts:6</code>); <code>GET /user</code> re-issues a token on every call (<code>user/index.get.ts:21</code>).
4	Unlimited password guessing — no rate limit or lockout Security BUG-04 · P5	1. Register an account. 2. Send 30 <code>POST /api/users/login</code> with wrong passwords. 3. Log in with the right password.	Expected: after ~5–10 failures: HTTP 429, delay or temporary lock (or CAPTCHA). Actual: 30 wrong guesses answered 401 in 3.7 s, never throttled; the right password still works at once. With #5, weak accounts can be guessed in seconds.	High	No rate-limiting or failed-attempt tracking on <code>users/login.post.ts</code> , and nothing in front of the API (gateway / WAF) that adds it.
5	Sign-up accepts an invalid e-mail and a 1-character password Validation · Security A4 · BUG-05 · P1–P2	1. Open “Sign up”. 2. Any username, e-mail <code>not-an-email</code> , password <code>a</code> . 3. Click “Sign up”.	Expected: inline errors: “enter a valid e-mail”, “password must be at least 8 characters”. Actual: account created and logged in. Inconsistent: changing the password in Settings <i>does</i> require 8+ characters.	High	Register schema only checks “not blank” (<code>user.schema.ts:5,7</code> — <code>min(1)</code> , no <code>.email()</code>) while the update schema enforces <code>min(8)</code> (<code>:25</code>). The form uses only <code>Validators.required</code> ; the e-mail input is <code>type="text"</code> .
6	“Not found” and loading errors show a blank white page Error handling · UX A3, A5 · BUG-06	1. Open <code>/article/this-article-does-not-exist</code> (or an old link from #2, or the profile from #9). 2. Repeat while the delete request fails (DevTools › block request / server error).	Expected: a friendly “This article doesn’t exist or was moved” message with a way back. Actual: empty page. The API <i>did</i> answer 404 <code>{"errors":{"article":["not found"]}}</code> , but the UI throws the message away.	Medium	Data-shape mismatch: <code>article.component.ts:73</code> and <code>profile.component.ts:46</code> store <code>err.errors</code> , but <code>ListErrorsComponent</code> reads <code>.errors</code> again (<code>list-errors.component.ts:13</code>), finds nothing and renders an empty list. Even the fallback message has the wrong shape.
7	“Delete Article” is one click with no confirmation — and a failed delete is silent UX · Error handling A6 · BUG-07a/b	1. Open one of your articles; click “Delete Article”. 2. Repeat while the delete request fails (DevTools › block request / server error).	Expected: “Delete this article?” confirmation (or Undo); on failure an error message and the button re-enabled. Actual: (1) deleted instantly and permanently. (2) nothing happens — no message; the user stays on the page not knowing why.	Medium	The button calls <code>deleteArticle()</code> directly (<code>article.component.html:26,73</code>), which subscribes with a success callback only (<code>article.component.ts:116</code>). “Disabled” is just a CSS class.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
8	E-mail treated as case-sensitive → “credentials invalid” and duplicate accounts Functional A7 · BUG-08 · P3	1. Sign up as Alice@example.com . 2. Log out; log in as alice@example.com with the right password. 3. Sign up again with alice@example.com .	Expected: login works (addresses are case-insensitive in practice); second sign-up rejected as “already taken”. Actual: login fails with “credentials invalid”; the lower-case address opens a second, separate account.	Medium	E-mail is stored exactly as typed and matched exactly (users/login.post.ts:11 ; SQLite compares case-sensitively). No normalisation (trim + lower-case) at sign-up or login.
9	Usernames containing ? , / or # make the user’s profile unreachable Validation A8 · BUG-09 · P4	1. Sign up with username qa?tester . 2. Click your name in the top bar.	Expected: profile opens (or sign-up rejects unsupported characters). Actual: the app asks the API for user “qa” (404) → blank page. / and # fail too; for ? and / even correctly encoded requests fail.	Medium	The API accepts any characters in usernames (user.schema.ts:6); the frontend builds URLs by string concatenation without encoding (profile.service.ts:19,27,33). Fix: allow e.g. [A-Za-z0-9_-] only and encode path segments.
10	No length or size limits: huge titles create unopenable articles and break the layout Validation · Performance A9–A10 · BUG-10 · P11–P12	1. New Article › paste a ~30,000-character title without spaces › Publish. 2. Go back to Home.	Expected: title limited (e.g. ≤ 200 chars) with a message; oversized requests rejected (413). Actual: saved (201) but can never be opened — the URL is ~30 KB and the API answers 431 → blank page. In the feed the title stretches the page ~330,000 px wide. A 10 MB article body is also accepted.	Medium	Schemas check min(1) but no .max() (article.schema.ts:5); the slug is built from the full title (articles/index.post.ts:13) and exceeds Node’s 16 KB request-header limit; no request-size limit; no CSS word-wrap for long words.
11	Accessibility: fails WCAG 2.1 AA checks on every page Accessibility A11 · BUG-11 (axe-core)	1. Run axe DevTools (or test BUG-11) on Home, Sign up, Article, Editor, Settings. 2. Try to delete your own comment with the keyboard only (Tab).	Expected: no serious/critical WCAG 2.1 AA violations; everything usable by keyboard. Actual: 82 low-contrast text instances, 17 images without alt text, 13 links without a name, no page language — on all 5 pages; the delete-comment icon can’t be reached by keyboard. Fields have placeholders but no visible labels.	Medium	Theme colours below the 4.5:1 contrast ratio; avatar without alt ; <html> without lang (index.html:2); a clickable <i class=“ion-trash-a”> instead of a <button aria-label=“Delete comment”> (article-comment.component.ts:33).
12	Invalid paging parameters crash the API (HTTP 500); page size is unlimited API robustness BUG-12 · P7, P7b	1. Open /api/articles?offset=-1 2. Open /api/articles?limit=100000	Expected: 400/422 “offset must be ≥ 0”; limit capped (e.g. 100). Actual: (1) 500 Server Error — the dev build even returns the internal database-query error text. (2) accepted: every article comes back in one response (performance / abuse risk as data grows).	Low	skip: Number(query.offset) 0 and take: Number(query.limit) 10 with no validation or clamping (articles/index.get.ts:22–23); the ORM error is not handled.
13	Tags aren’t trimmed: “ news ” and “news” become two identical-looking tags Data quality A12 · BUG-13 · P10	1. New Article › type news (with spaces) as a tag › Enter › Publish. 2. Publish another article tagged news ; look at “Popular Tags”.	Expected: one tag “news”. Actual: two separate tags that look identical; filtering by “news” misses the article whose tag has spaces.	Low	The editor checks tag.trim() but stores the untrimmed value (editor.component.ts:63); the API accepts any string (article.schema.ts:8).

4. Root-cause analysis — #2 “Editing an article changes its web address”

What happens. Every time an author saves an article — even to fix a single typo in the body — the article gets a new address (in screenshots A2a → A2b, `/article/Our-release-notes-fc56afda` became `/article/Our-release-notes-cb171958`), and the old address answers 404, which the UI then shows as a blank page (#6).

Why. The API's update handler creates a brand-new slug with a random 8-character suffix whenever the request contains a `title` field (`articles/[slug]/index.put.ts:35`) — it never checks whether the title actually changed. The Angular editor always submits the whole form — title, description, body and tags (`editor.component.ts:80`) — so a title is present on every save and a new slug is minted every time. The RealWorld documentation only says the slug is updated “when the title is changed”, and the project's own API test updates the body *without* sending a title, which is why the automated suite never caught it: the bug only appears when this frontend is combined with this backend. Nothing keeps the previous slug, so every link shared on social media, in e-mails or indexed by search engines silently dies after each edit — a high-frequency problem, since editing is routine.

How I would fix it. (1) In the API, only mint a new slug when the new title differs from the stored one — a 3-line change that I applied locally and verified (below). (2) Longer term, keep a slug-history table and answer old slugs with a 301 redirect, so even real title changes never break links. (3) Have the editor send only the fields that changed. (4) Turn my Playwright reproduction (BUG-02) into a regression test in CI — against the patched API it now fails exactly at “the URL changed although the title did not”, which proves the fix.

```
// server/routes/api/articles/[slug]/index.put.ts (full diff: evidence/fix-bug02-stable-slug.diff)
select: {
+   title: true,
+   author: { select: { id: true, username: true } },
-   const newSlug = article.title ? `${slugify(article.title)}-${crypto.randomUUID().slice(0, 8)}` : null;
+   // Only mint a new slug when the title really changes (docs: "the slug also gets updated when the title is changed").
+   const titleChanged = article.title !== undefined && article.title !== existingArticle.title;
+   const newSlug = titleChanged ? `${slugify(article.title!)}-${crypto.randomUUID().slice(0, 8)}` : null;
```

Fix verification (evidence/fix-verification-bug02.txt): body-only edit → slug unchanged ✓ · old link → HTTP 200 ✓ · genuine title change → slug still updated, as documented ✓ · reproduction test BUG-02 now fails at the “URL changed” assertion ✓. The patch was then reverted so all other results reflect the original app.

5. What works well (verified, not assumed)

- ✓ All core flows — sign-up, log-in, create, edit, comment, delete, log-out — pass the automated happy-path test.
- ✓ Stored XSS is neutralised: `<script>`, `onerror=` and `javascript:` links in an article body do not execute (Angular sanitizer).
- ✓ Authorization holds: another user gets 403 when editing or deleting my article or deleting my comment, and the content stays unchanged (probe P15).
- ✓ Duplicate tags inside one article are merged into one (probe P9).

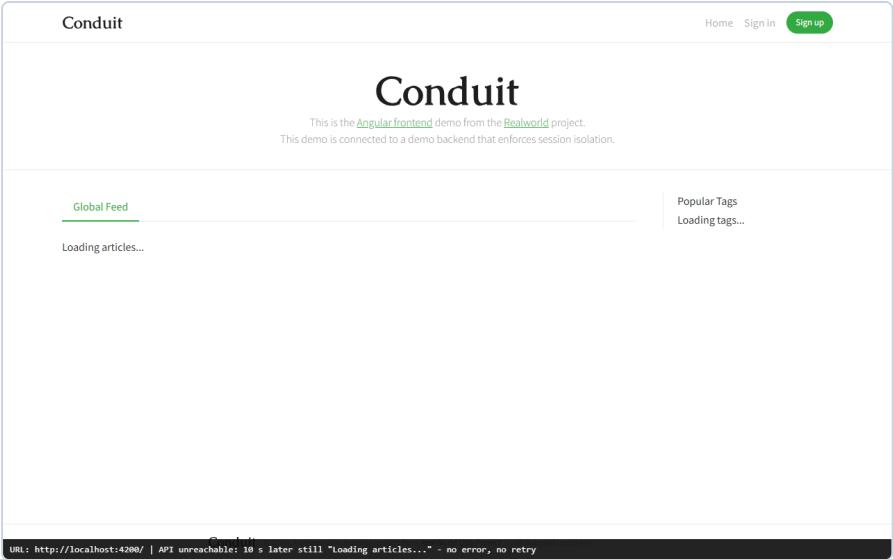
6. Other observations (not counted above)

- A user can follow themselves through the API (the UI hides the button) — probe P14.
- The API address is hard-coded in the frontend source (`api.interceptor.ts`), so switching environments needs a code change instead of configuration.
- Article and profile pages render nothing while data is loading — no spinner or skeleton, so slow connections look like #6.
- The brief's live-demo link is outdated: `demo.realworld.io` returns 404; the project now lives at `demo.realworld.show`.

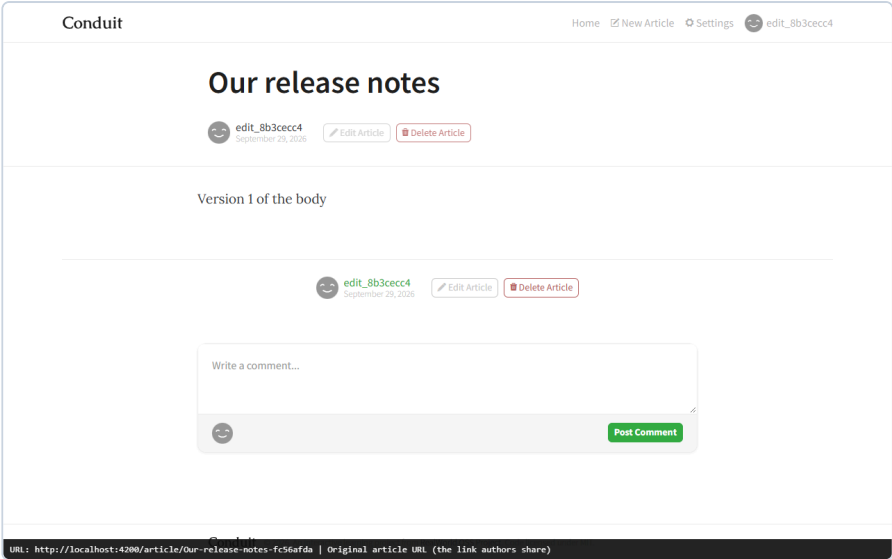
7. How to reproduce everything

```
powershell -ExecutionPolicy Bypass -File setup\setup-realworld.ps1 # once: pinned commits, install, create DB
powershell -ExecutionPolicy Bypass -File setup\start-realworld.ps1 -ResetDb # fresh, empty app on :4200 / :3000
cd task1-qa-report\qa-tests ; npm install ; node api-probes.mjs ; npx playwright test # evidence + 16 tests
```

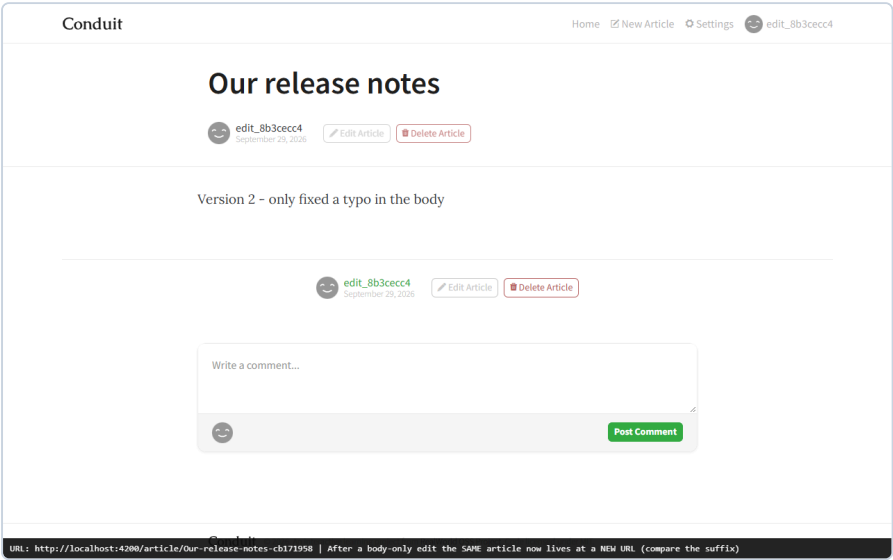
Appendix A — Screenshots (captured automatically by the Playwright tests; the dark bar shows the page URL)



A1 · #1 API unreachable: after 10 s the feed and tags still say “Loading...”. No error, no retry.



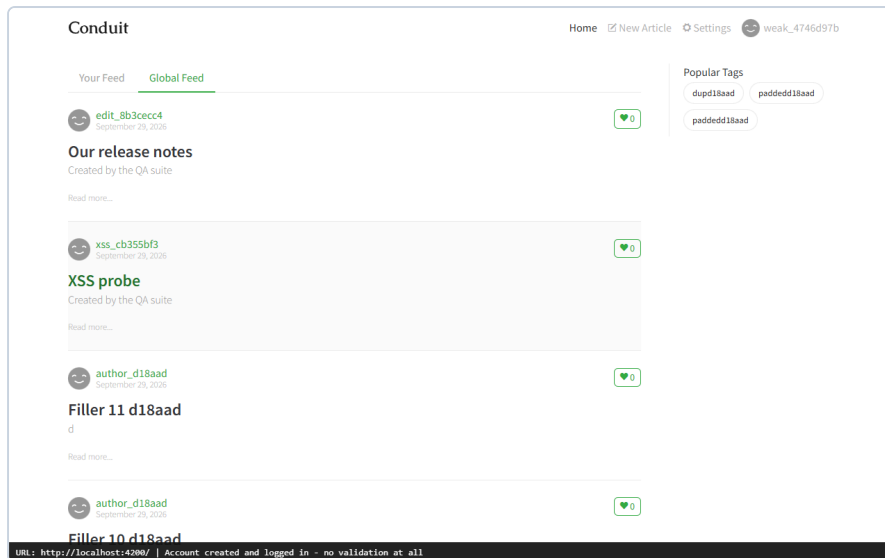
A2a · #2 The article at its original URL (the link an author would share).



A2b · #2 After changing only the body, the same article lives at a **new** URL (different suffix).



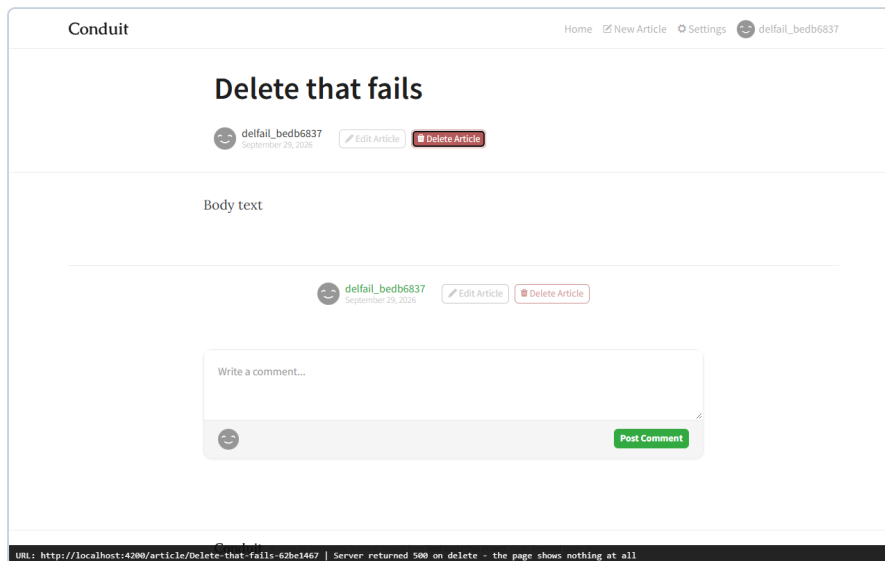
A3 · #2 + #6 Opening the original link now: API 404, and the user sees a blank page.



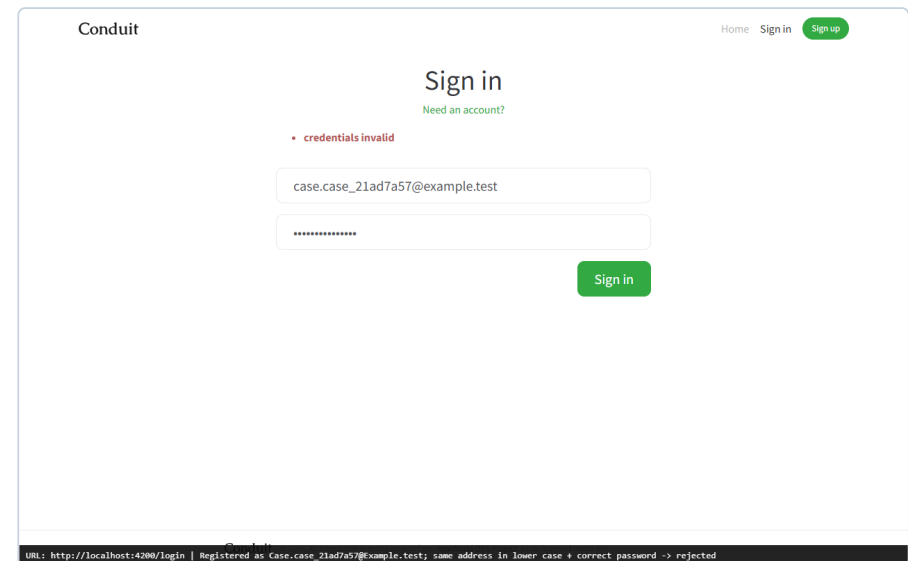
A4 · #5 Account created with e-mail “not-an-email-...” and password “a”, and logged straight in.



A5 · #6 Non-existent article: the API sends “article not found”, the page shows nothing.



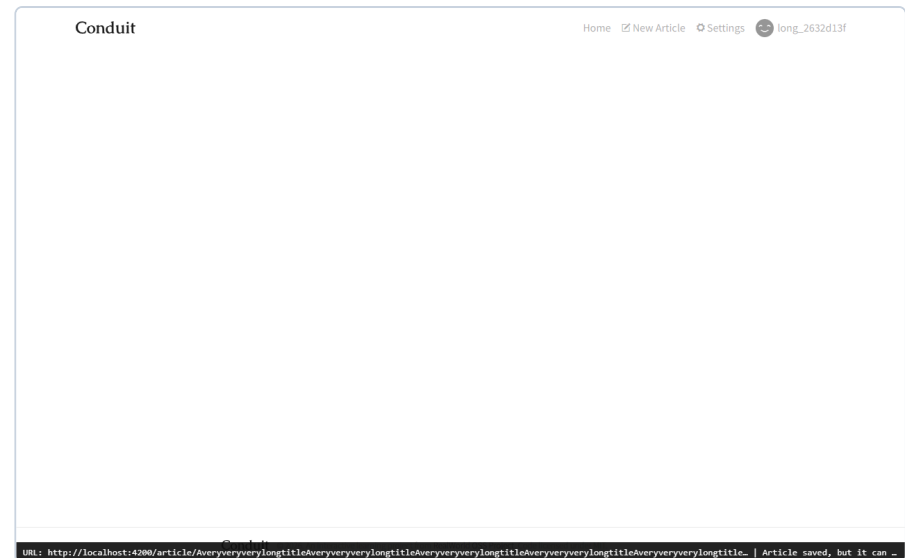
A6 · #7 Server error on delete: no message at all; the user stays on the page. (Without an error, deletion is instant and permanent — no confirmation.)



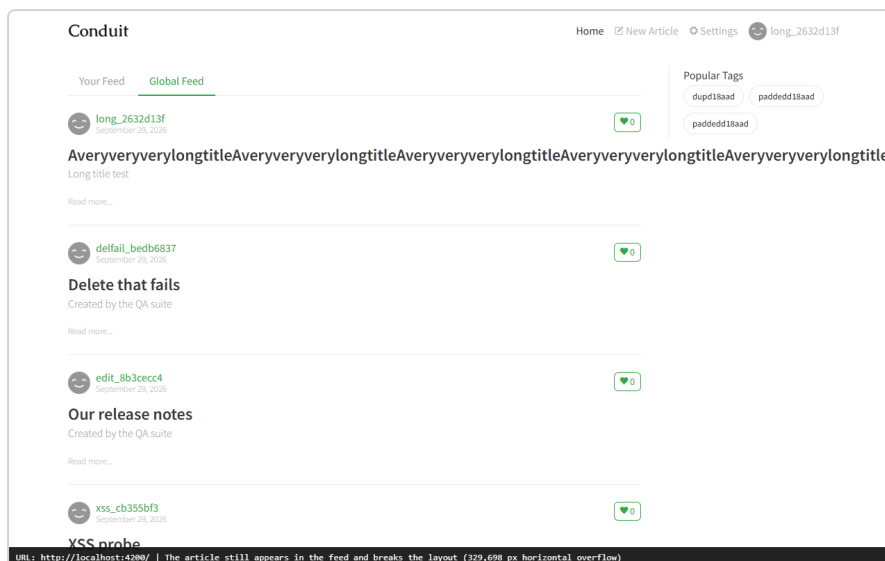
A7 · #8 Registered with capital letters; the same address in lower case + correct password → “credentials invalid”.



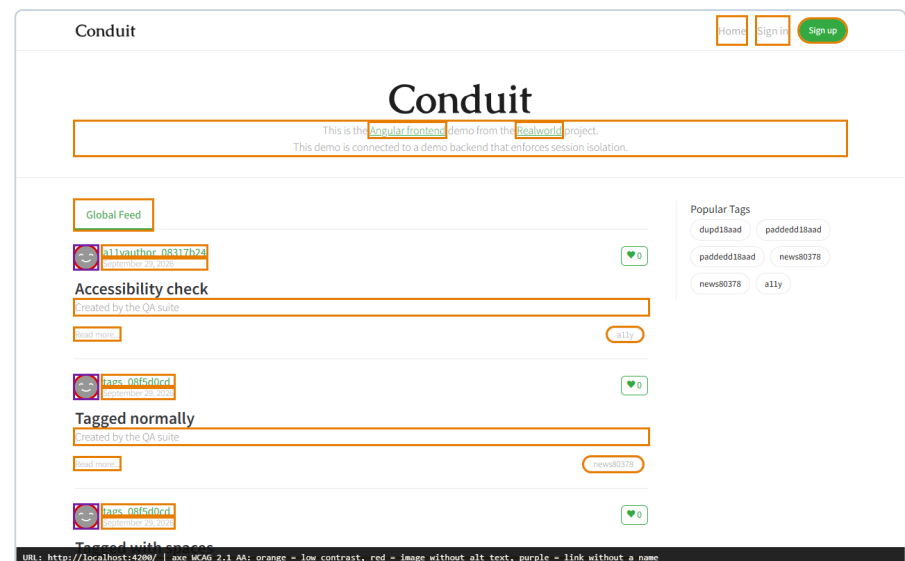
A8 · #9 User “qa?tester_...” clicks their own name: the app requests user “qa” (404) and shows a blank page.



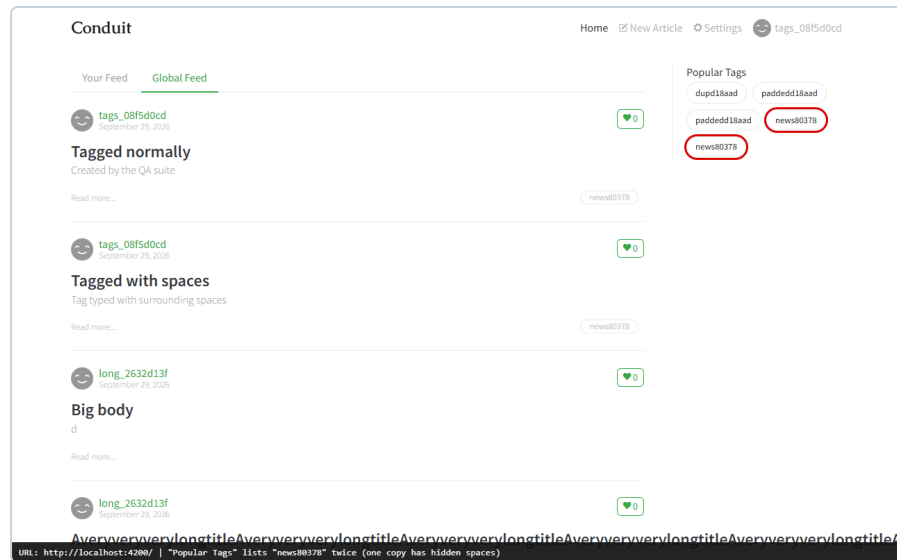
A9 · #10 A ~30,000-character title is saved (201) but the article can never be opened (431) → blank page.



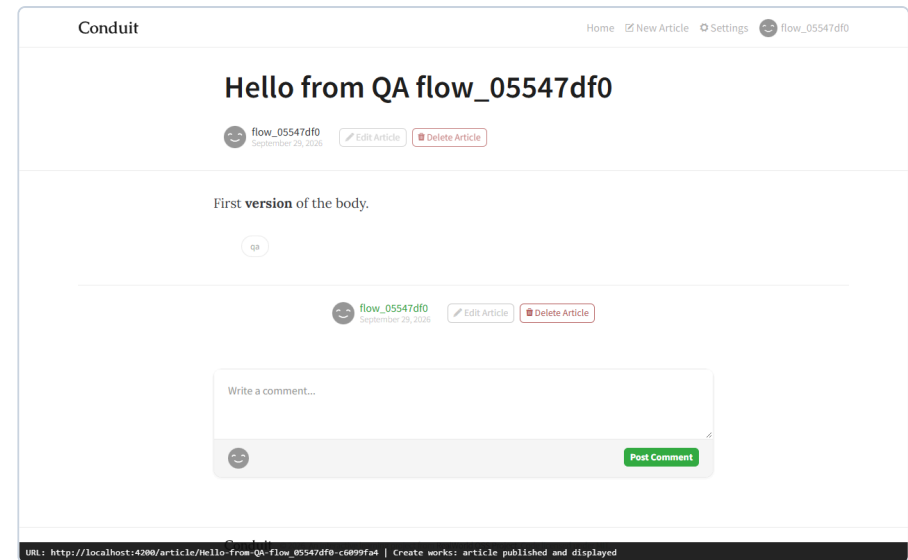
A10 · #10 The same article in the feed: the unbroken title runs off the page (~330,000 px horizontal overflow).



A11 · #11 axe-core findings on Home: orange = low contrast, red = image without alt text, purple = link without a name.



A12 • #13 "Popular Tags" lists the same-looking tag twice — one copy was typed with spaces.



A13 • passes For contrast: the normal create flow works — the article is published and displayed.